Real-Time Smart Home Hazard and Intrusion Detection on Raspberry Pi 5: Deployment and a Layer-Level CPU, GPU and FPGA Comparison

Narendranaath S R[1], Anwesha Bhuyan[1], Melwin Biju[1], and Sairabanu J[1]

[1] School of Computer Science and Engineering, Vellore Institute of Technology, India
narendranaath.sr2026@vitstudent.ac.in,
anwesha.bhuyan2026@vitstudent.ac.in,
melwin.bijukalayil2026@vitstudent.ac.in, jsairabanu@vit.ac.in

Abstract. Cloud-connected home cameras introduce delay, depend on a stable network, and expose private video. This paper presents a smart home hazard and intrusion detection system that runs entirely on an edge device. It also examines, from a computer-architecture perspective, how a single convolution layer of the detector behaves on a CPU, a GPU and an FPGA. We trained an eight-class YOLO11n detector for 100 epochs on a rebalanced dataset. On the validation set it reaches a precision of 0.918, a recall of 0.851 and an mAP@0.5 of 0.887. On a Raspberry Pi 5, INT8 inference with OpenVINO managed only about 0.1 frames per second (FPS), since this runtime has no fast INT8 path on ARM processors. Exporting the model to NCNN with a 480×480 input raised the rate to roughly 18–20 FPS with live display. For the hardware study, we wrote the first convolution layer in Verilog as a circuit with 16 multiply-accumulate (MAC) units and simulated it with Icarus Verilog. The circuit needs 2,764,827 clock cycles (27 per output pixel), which is 27.6 ms at 100 MHz, and all 1,638,400 outputs match an integer reference model. The same layer takes 6.0 ms on an Apple M1 CPU with OpenVINO and 5.0 ms on the M1 GPU with PyTorch. Both agree with the circuit on 99.93% of values, and the remainder differ by a single rounding step. The circuit is slower only because it has few parallel units; with 432 MAC units, the layer would take about 1 ms.

Keywords: Edge AI · YOLO11 · Raspberry Pi 5 · NCNN · INT8 quantization · FPGA · Verilog · Smart home surveillance

1 Introduction

Fire, smoke, intruders and snakes are serious risks in and around a home. Conventional protection relies on single-purpose sensors such as smoke detectors and motion sensors, which cannot tell a real threat from a harmless event, so false alarms remain a central problem [1]. Camera-based detection with convolutional neural networks (CNNs) addresses this. CNNs can detect fire in surveillance video [2], YOLO detectors find smoke and wildfire [3], and lightweight fire detectors run on low-power devices [4, 5]. Related home applications include parcel delivery detection [6], pet monitoring [7] and access control [8].

One-stage detectors of the YOLO family [9] made real-time detection practical, and the Ultralytics YOLO11 models [10] include a nano version with 2.6 million

parameters. For a home device, though, the main difficulty is execution rather than accuracy. Every frame needs billions of MAC operations, and how quickly a processor performs them depends on its parallel units, its number formats and its memory system [11]. Using 8-bit integers (INT8) instead of 32-bit floating point makes models smaller and, on suitable hardware, faster [12, 13]. Intel OpenVINO with its NNCF quantization tool [14] is widely used for this on Intel processors [15-17]. Low-cost edge boards such as the Raspberry Pi 5 [18], however, use ARM processors, for which runtimes such as NCNN [19] are designed.

FPGAs offer a third option. Instead of executing instructions, they are configured into a circuit that performs many MAC operations in parallel [20-24]. Intel's current tool for running OpenVINO models on FPGAs, the FPGA AI Suite, needs a licence and an Intel FPGA board [25]. We therefore wrote our own circuit for the first layer of the model in Verilog and tested it with the Icarus Verilog simulator [26].

A fire or an intruder calls for an alert within seconds, without relying on an internet connection, and video from inside a home should not leave the home. A small detector running on a single-board computer meets these needs. Studying one layer on a CPU, a GPU and an FPGA shows which hardware properties decide the speed. Table 1 summarises the feasibility of the system.

Table 1. Feasibility of the proposed system

Aspect	**Justification**
Technical	The model has 2.6 million parameters (5.5 MB file) and runs at about 18–20 FPS on a Raspberry Pi 5 with NCNN; all software used is open source.
Economic	A Raspberry Pi 5 and a webcam replace a cloud service or a GPU server; no paid software is needed.
Social	Local alerts for fire, smoke, intruders and snakes help families, elderly people and pet owners; cats and dogs are separate classes, which reduces false alarms.
Environmental	No continuous video upload to data centres; a small model on a single-board computer instead of a server.
Political / legal	Video stays on the device and only alert snapshots are e-mailed, in line with data-protection expectations such as India's Digital Personal Data Protection Act, 2023.
Demographic	Works offline, so it suits rural and low-bandwidth areas; alerts reach any phone by e-mail.

Our objective is to build a smart home hazard and intrusion detection system that runs fully on a Raspberry Pi 5, and to check how its first convolution layer performs on a CPU, a GPU and an FPGA.

This work makes four contributions. (1) We present a complete edge system on a Raspberry Pi 5, covering a rebalanced dataset, a YOLO11n model trained for 100 epochs, and NCNN deployment at 480×480 with geofence and time rules and threaded e-mail alerts. (2) We report measurements on two ARM devices showing that OpenVINO INT8 is not a fast path on ARM processors. (3) We provide a Verilog implementation of the first convolution layer that uses the exact INT8 weights of the

OpenVINO model and is verified value by value in simulation. (4) We give a measured comparison of this layer on a CPU, a GPU and the simulated FPGA, using the same image and including how closely their outputs agree.

2 Literature Review

We reviewed 20 works from reputed journals and conferences, grouped into four areas: detectors and quantization [9, 12, 13]; hardware for neural networks [11, 20-24]; home-safety applications [1-8]; and edge deployment with OpenVINO [15-17]. Table 2 lists, for each work, the problem it addresses, its method, result, dataset and weaknesses.

Table 2. Summary of the 20 reviewed works

Ref.	Problem	Method	Result	Dataset	Weaknesses
[9]	Slow multi-stage object detection	Single CNN regresses boxes and class scores in one pass	45 FPS (base), 155 FPS (Fast YOLO)	PASCAL VOC	Lower localisation accuracy than two-stage detectors
[12]	Floating-point cost on mobile CPUs	Affine 8-bit quantization $r = S(q - Z)$ with quantization-aware training	Better accuracy–latency trade-off for MobileNets	ImageNet, COCO	Needs (re)training for best accuracy
[13]	DNN acceleration by compression	Survey of static/dynamic pruning and quantization	Taxonomy and comparison of reported trade-offs	Literature	No new measurements
[11]	Energy/throughput of DNN hardware	Tutorial on dataflows, reduced precision, metrics	Framework for evaluating accelerators	Literature	Pre-dates YOLO11-class models
[20]	Data movement energy in CNN accelerators	Row-stationary dataflow, 168-PE array	AlexNet conv layers 34.7 fps at 278 mW (65 nm)	AlexNet, VGG-16	ASIC, not reprogrammable
[21]	Mapping CNNs onto embedded FPGAs	Quantization to 8 bit + compiler + parameterised hardware	~6× faster than a peer FPGA design on XC7Z045	VGG, detection tasks	Vendor tools, fixed architecture
[22]	Loop/dataflow choices for conv on FPGA	Loop unrolling/tiling/interchange analysis	VGG-16: 348 GOPS (Stratix V), 715 GOPS (Arria 10)	NiN, VGG-16, ResNet	Large FPGAs; 16-bit data
[23]	Real-time YOLO on FPGA	Binary weights, low-bit activations, streaming design	1.877 TOPS at 200 MHz, 18.29 W; 64.16% mAP	PASCAL VOC 2007	2.63-point mAP loss; retraining
[24]	Organising FPGA CNN designs	Survey by optimisation technique	Classification of >100 works	Literature	No experiments
[2]	Fire detection in surveillance video	SqueezeNet-style CNN, small kernels, no FC layers	Accuracy comparable to larger CNNs at lower cost	Fire video benchmarks	Classification-centric
[4]	Fire/smoke de-	CNN detector + filter	Up to 83.60%	D-Fire	Pruning needs

Ref.	Problem	Method	Result	Dataset	Weaknesses
	tection without GPUs	pruning	lower compute, 83.86% lower memory		retraining
[3]	Choosing a YOLO version for smoke	YOLOv5–v8 and YOLO-NAS compared	No single best model; recall vs. precision trade-off	Foggia + custom set	Accuracy only; no deployment
[5]	Fire detection in aerial imagery	MobileNet + soft attention + 3D convolution	Higher accuracy than prior methods	FD, DFAN, ADSF	Aerial focus
[1]	False alarms and datasets in fire detection	Review of CNN, YOLO, spatiotemporal models	Identifies dataset and false-alarm gaps	Literature	No experiments
[7]	Pets in surveillance video	YOLOv3 + Kalman/ CNN tracking + emotion CNN	Detection 97.59%/94.62%, tracking up to 93.02%	Own dog videos	Several heavy CNNs
[8]	Smart-home access control	Iris recognition + FisherFace emotion detection	Iris 95.25%, emotion 93.93% accuracy	Own data	No hazard detection
[6]	Parcel delivery events	Motion proposals + mobile 3D CNN	Outperforms alternatives (no figures in abstract)	Curated delivery set	Single task
[16]	Low-cost edge deployment	OpenVINO person detection on several devices	Deployment judged promising	—	No ISA-level analysis
[17]	Edge speed and power	OpenVINO 2022.3 on Pi 4B with NCS2	~400% speed-up (>50 FPS) for classification	Own images/video	NCS2 no longer supported by current OpenVINO
[15]	Precision effects in OpenVINO	14 classifiers in FP32/FP16/INT8	INT8 1.02–1.56× on CPU; FP16 1.1–2× on GPU	ImageNet models	Classification only; older CPU

Three observations stand out. Application papers report accuracy but rarely say how fast the model runs on real low-cost hardware [3, 5, 7]. FPGA detector designs reach high throughput, but they depend on retrained custom number formats and large devices [22, 23]. Deployment papers compare devices, yet the benefit of INT8 varies strongly with the processor [15].

2.1 Research Gaps

- **G1 – Single-purpose systems.** Most works detect only one kind of event, such as fire, parcels or pets [3, 6, 7]. *This work:* a single eight-class detector with alert rules for fire and intrusion.
- **G2 – Accuracy without deployment.** Detection papers rarely demonstrate real-time execution on a low-cost edge board [3, 5]. *This work:* the final model runs on a Raspberry Pi 5 at about 18–20 FPS.

- **G3 – Runtime and processor mismatch.** INT8 speed-ups depend on hardware support [15]. *This work:* we show on two ARM devices that OpenVINO INT8 is slow, and use NCNN instead.
- **G4 – FPGA designs need special models.** FPGA detectors often retrain with custom low-bit weights on large boards [21, 23], and Intel's route needs a licence [25]. *This work:* our circuit uses the exact INT8 weights of a standard OpenVINO model and is checked value by value.
- **G5 – No layer-level comparison.** Hardware comparisons usually report whole models. *This work:* one layer is compared on a CPU, a GPU and an FPGA with the same input, including output agreement.

3 Materials and Methodology

3.1 Overview of the Proposed Work

Fig. 1 shows the proposed work, which has two parts. Part A is the deployed system. A dataset prepared on Roboflow is used to train a YOLO11n model on a GPU workstation (best_5.pt), which is exported to NCNN with a 480×480 input. On a Raspberry Pi 5 with a webcam, the model then performs real-time detection, followed by alert rules and e-mail alerts. Part B is the hardware study. The first convolution layer of the model is run on the CPU and GPU of an Apple M1 laptop and as a Verilog circuit in simulation, and the times and outputs are compared.

Part B uses the INT8 OpenVINO version (best.xml and best.bin, 640×640 input) of our first YOLO11n model, because the circuit needs OpenVINO's INT8 weights and scale values. The final model, best_5.pt, has exactly the same first layer (16 filters of size 3×3×3, stride 2, padding 1), which we verified from the checkpoint. Only the 432 weight values differ, so the circuit applies to the final model without change.

Fig. 2 is a simple block diagram of the deployed system, and Fig. 1 shows the detailed architecture. Frames from the webcam are resized to 480×480 and passed through YOLO11n in NCNN on the Raspberry Pi 5. The outputs are decoded into boxes and checked by the alert rules. If a hazard is found, an e-mail with a snapshot is sent.

3.2 Dataset and Training Configuration

The dataset was assembled on the Roboflow platform and exported in YOLO format. To reduce class imbalance, we capped the Fire class, which originally had more than 18,000 images. The Cat and Dog classes were kept as non-hazard classes so that the model learns to tell pets apart from hazards and intruders. The class list stored in the trained model has eight classes: Cat, Dog, Fire, Intrusion, Package, Person, Smoke and Snake. Table 3 lists the training settings exactly as stored in best_5.pt. Training ran on a workstation with an Intel Core i9-14900K processor and an NVIDIA RTX A4000 GPU (16 GB).

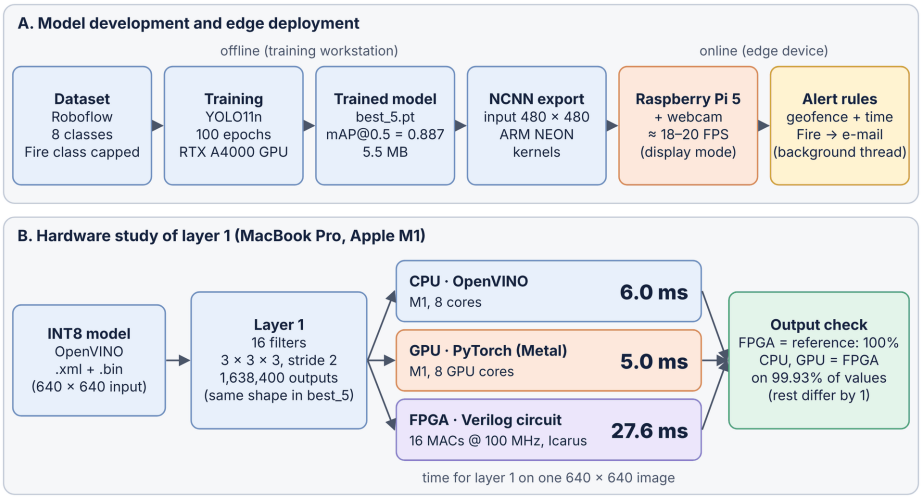


Fig. 1. Architecture of the proposed work. A: model development and deployment on the Raspberry Pi 5. B: hardware study of the first convolution layer on the CPU, the GPU and a simulated FPGA circuit (measured times for one 640×640 image).

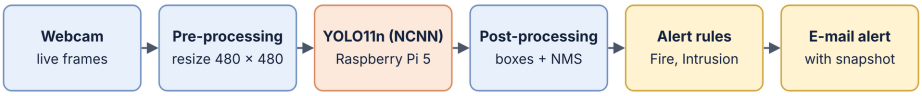


Fig. 2. Simple block diagram of the deployed system.

3.3 Test Images

The hardware study uses one image from the D-Fire dataset [4], letterboxed to 640×640. The detection examples use D-Fire images, public ImageNet [27] example images of snakes and a cat, and a standard sample image of a dog.

3.4 Dataset Rebalancing and Augmentation

Besides capping the Fire class, the training pipeline creates new variations of each image on the fly. Mosaic combines four images into one, which shows objects in new contexts and at different scales. It is switched off for the last 10 epochs so the model finishes training on natural images. Colour shifts in HSV space imitate the poor exposure of cheap webcams. The gains are g = 1 + u, where u is drawn uniformly from [−a, a] and a = (0.015, 0.7, 0.4) for hue, saturation and value, as in Eq. (1):

$$H' = \left(H\, g_h\right) \bmod 180,\; S' = \min\left(255, S\, g_s\right),\; V' = \min\left(255, V\, g_v\right) \quad (1)$$

Random translation (±10%), scaling (±50%), horizontal flips (probability 0.5), RandAugment and random erasing (probability 0.4) complete the augmentation [10].

Table 3. Training configuration (stored in best_5.pt)

Item	Value
Base model / framework	YOLO11n (pre-trained yolo11n.pt) / Ultralytics 8.4.177
Classes	8 (Cat, Dog, Fire, Intrusion, Package, Person, Smoke, Snake)
Input size / batch / workers	640×640 / 64 / 16
Epochs	100 (early-stopping patience 25), mixed precision
Optimizer	MuSGD, learning rate 0.01 → 0.0001, momentum 0.937, weight decay 5×10^{-4}, 3 warm-up epochs
Loss weights	box 7.5, classification 0.5, distribution focal loss 1.5
Augmentation	mosaic (off in the last 10 epochs), HSV shifts (h 0.015, s 0.7, v 0.4), translate 0.1, scale 0.5, horizontal flip 0.5, RandAugment, random erasing 0.4
Training time	10.6 h (6.3 min per epoch)
Parameters / file size	2,606,584 / 5.52 MB

3.5 YOLO11n Network

YOLO11n has four main parts: a backbone of strided convolutions and C3k2 blocks, an SPPF (spatial pyramid pooling, fast) block that enlarges the receptive field, a neck that merges three scales, and a detection head [10]. Almost all of the work is convolution. For output channel c at position (i, j), with stride s and padding p, as in Eq. (2):

$$y_{c,i,j} = b_c + \sum_{m} \sum_{u} \sum_{v} w_{c,m,u,v}\, x_{m,si+u-p,sj+v-p} \tag{2}$$

Each term is one multiply-accumulate (MAC) operation. After every convolution, the SiLU activation is applied [28], as in Eq. (3):

$$\mathrm{SiLU}(z) = z\,\sigma(z) = \frac{z}{1+e^{-z}} \tag{3}$$

For an input of S×S pixels, the head predicts $(S/8)^2 + (S/16)^2 + (S/32)^2$ candidate boxes, each with 4 box values and 8 class scores. That gives 8,400 boxes at 640×640 and 4,725 at 480×480. The work of every layer grows with the image area, so the 480×480 input of the edge device needs

$$\mathrm{MACs}_{480} = \mathrm{MACs}_{640} \cdot \left(\frac{480}{640}\right)^2 = 0.5625 \cdot \mathrm{MACs}_{640} \tag{4}$$

of the operations, or about 3.6 instead of 6.4 GFLOPs per frame.

3.6 Training Objective and Metrics

The detector is trained with a weighted sum of three terms: a box-regression loss (CIoU [29]), a classification loss (binary cross-entropy) and a distribution focal loss for the box edges [30]. The weights in Eq. (5) come from Table 3:

$$L = \lambda_{\text{box}} L_{\text{CIoU}} + \lambda_{\text{cls}} L_{\text{BCE}} + \lambda_{\text{dfl}} L_{\text{DFL}}, (\lambda_{\text{box}}, \lambda_{\text{cls}}, \lambda_{\text{dfl}}) = (7.5, 0.5, 1.5) \quad (5)$$

A predicted box counts as correct when it overlaps a labelled box enough, as measured by the intersection over union in Eq. (6):

$$\text{IoU}(A, B) = \frac{|A \cap B)}{|A \cup B)} \quad (6)$$

Precision and recall are P = TP/(TP + FP) and R = TP/(TP + FN). The average precision (AP) of a class is the area under its precision–recall curve, and the mean average precision (mAP) is the mean over the N classes, as in Eq. (7):

$$\text{AP} = \int_0^1 p(r)\, dr, \text{mAP} = \frac{1}{N} \sum_{k=1}^{N} \text{AP}_k \quad (7)$$

mAP@0.5 uses an IoU threshold of 0.5, while mAP@0.5:0.95 averages the results over ten thresholds from 0.5 to 0.95.

3.7 Edge Deployment on the Raspberry Pi 5

Our first deployment attempt used OpenVINO with INT8 post-training quantization [14], which maps a real value x to an 8-bit integer q using a scale s and a zero point z [12], as in Eq. (8):

$$q = \text{clamp}(\text{round}(x/s) + z, -128, 127), x \approx s(q - z) \quad (8)$$

On the Raspberry Pi 5 this ran at only about 0.1 FPS. On ARM processors, OpenVINO does not run the quantized layers with fast integer instructions but computes them in floating point, so the extra quantize and de-quantize steps only add work. We therefore moved the deployment to NCNN [19], a C++ inference framework for mobile and embedded processors that has no third-party dependencies. Its kernels are hand-optimised for the ARM NEON vector unit and use several cores. We exported the PyTorch weights to NCNN with a 480×480 input (Eq. (4)), which gives about 18–20 FPS with the live display.

3.8 Application Logic and Alerts

The application reads webcam frames with OpenCV and runs in one of two modes: a display mode that draws boxes and the frame rate, or a headless mode without a window. The Intrusion class is difficult to learn from images alone, so a Person detection is turned into an intrusion when its reference point c lies inside a virtual geofence G during a configured time window T, as in Eq. (9):

$$\text{intrusion} = 1[\text{class} = \text{Person}) \cdot 1[c \in G) \cdot 1[t \in T) \quad (9)$$

When a critical hazard such as fire is detected, the current frame is encoded as a JPEG image, attached to an e-mail as a MIME image and sent with Python's smtplib. Sending an e-mail can take seconds, so it runs in a separate background thread and the video loop never waits for the network.

3.9 FPGA Circuit for the First Layer

Layer 1 of the model is a convolution with 16 filters of size 3×3 over the 3 colour channels, with stride 2 and SiLU. A Python script reads its 432 INT8 weights (16 × 3 × 3 × 3) and the scale values from the OpenVINO files best.xml and best.bin, and writes them, together with the test image, into memory files that the circuit loads. A 256-entry table maps the 8-bit input pixels to the quantized input values.

For every output position, the 27 values of the 3×3 window in 3 colours are read one per clock cycle. The circuit has 16 MAC units, one per filter. They all receive the same input value and multiply it by their own weight, as in Eq. (10):

$$\text{acc}_c \leftarrow \text{acc}_c + q_x w_c, \text{repeated 27 times, for } c = 0, \ldots, 15 \text{ in parallel} \quad (10)$$

The 24-bit sum is converted back to 8 bits with a fixed-point multiplier M_c and bias B_c, and a 65,536-entry table applies SiLU and the output quantization in a single step, as in Eq. (11):

$$\text{idx} = \text{floor}\left(\frac{\text{acc}_c M_c + B_c + 2^{23}}{2^{24}}\right) + 24576, q_y = \text{LUT}[\text{idx}) \quad (11)$$

The circuit is pipelined, so a new input enters every cycle and 16 outputs are produced every 27 cycles. For a 640×640 image, layer 1 has 320 × 320 = 102,400 output positions. The number of cycles and the time at clock frequency f are

$$\text{cycles} = 102{,}400 \times 27 + 27 = 2{,}764{,}827, \text{time} = \text{cycles}/f \quad (12)$$

where the extra 27 cycles fill the pipeline. More generally, a layer with a given number of MAC operations on P parallel MAC units takes

$$\text{time} = \frac{\text{MACs}}{Pf} = \frac{44{,}236{,}800}{16 \times 100 \times 10^6} = 27.6\,\text{ms} \quad (13)$$

Fig. 3 shows the circuit. A Python program computes the same layer with integer arithmetic (the golden reference model), and every output of the simulation is compared against it.

3.10 Implementation on Each Platform

Table 4 summarises how the model and the layer were run on each platform. The CPU, GPU and FPGA results were measured on the same MacBook Pro with the Apple M1 chip, using the same input image and the same INT8 weights.

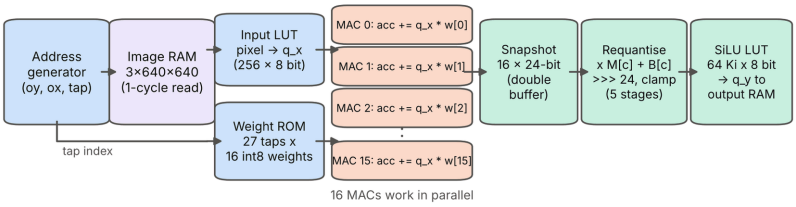


Fig. 3. Block diagram of the layer-1 FPGA circuit with 16 MAC units working in parallel.

Table 4. Implementation on each platform

Platform	Hardware	Software	Work and precision	Measurement
CPU	Apple M1, 8 cores	OpenVINO Runtime	Layer 1 cut from the INT8 model; whole model in FP32, FP16, INT8	Median of 100 runs (layer 1); mean of 30 runs (whole model)
GPU	Apple M1, 8 GPU cores	PyTorch on Metal (MPS)	Layer 1 with the same INT8 weights and quantization steps in float32; whole model in FP32, FP16	Median of 100 runs, including copies to and from the GPU
FPGA (simulated)	Our Verilog circuit, 16 MAC units, 100 MHz clock	Icarus Verilog 13.0	Layer 1, integer arithmetic only	Clock cycles counted by the testbench; time = cycles / f
Edge device	Raspberry Pi 5 with webcam	NCNN, OpenCV, Python	Whole model best_5.pt, 480×480 input	Frames per second in display mode

- **CPU:** Layer 1 is cut out of the INT8 model (input quantization, convolution, bias, SiLU and output quantization) and run with OpenVINO in float32 arithmetic.
- **GPU:** OpenVINO's GPU plugin supports Intel GPUs only, so the M1 GPU is used through PyTorch with Apple's Metal backend. The layer uses the INT8 weights multiplied by their scales and the same quantization formulas as OpenVINO, so its output can be compared with the other two.
- **FPGA:** No FPGA board was used. The simulator executes the Verilog design clock cycle by clock cycle, exactly as the hardware would behave, and counts the cycles. The time then follows from Eq. (12) with f = 100 MHz, a realistic clock for small, low-cost FPGAs.

3.11 Performance Parameters

The parameters we used to evaluate the system are listed in Table 5.

Table 5. Performance parameters considered

Parameter	Meaning (where used)
Precision, recall, mAP@0.5, mAP@0.5:0.95	Detection accuracy on the validation set, Eq. (7) (model)
Frames per second (FPS)	Frames processed per second, FPS = 1000 / latency in ms (edge device, laptop)
Latency	Time per frame or per layer in ms (laptop)
Clock cycles, cycles per output pixel	Work done by the circuit, counted in simulation (FPGA)
Time at clock frequency	time = cycles / f, f = 100 MHz (FPGA)
Output agreement	Share of the 1,638,400 layer-1 outputs that are identical, and the largest difference (CPU, GPU, FPGA)
Effective MAC rate	MAC operations per second achieved on layer 1 (CPU, GPU, FPGA)

3.12 Software Environment

Laptop (macOS, Apple M1):

1. Create a Python environment: `python3 -m venv ~/openvino_envi`, then `source ~/openvino_envi/bin/activate`.
2. Install the libraries: `pip install -r requirements.txt` (OpenVINO, OpenCV, NumPy, PyYAML, Ultralytics with PyTorch).
3. Install the Verilog simulator with `brew install icarus-verilog` (check it with `iverilog -V`; we used version 13.0).
4. Run the FPGA simulation with `bash fpga/sim/run_sim.sh`. To compare layer 1 on the FPGA, CPU and GPU, run `python3 fpga/sim/compare_all.py`.

Edge device (Raspberry Pi 5):

1. Install Python with Ultralytics, NCNN and OpenCV.
2. Export the model with `yolo export model=best_5.pt format=ncnn imgsz=480`.
3. Connect the webcam and run the detection script in display mode (OpenCV window) or headless mode. Set the e-mail account and recipient for alerts.

4 Experimental Results and Discussion

4.1 Detection Accuracy

Table 6 compares the final model with our first model, and Fig. 4 shows the training history stored in best_5.pt. The best epoch was 97.

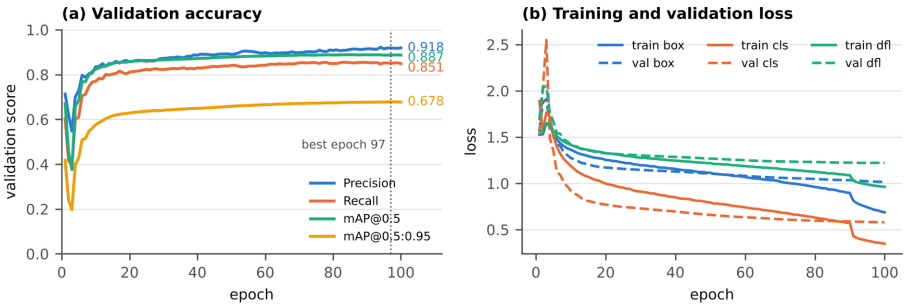


Fig. 4. Training history of the final model (best_5.pt): (a) validation precision, recall and mAP, with the values of the best epoch; (b) training and validation losses.

Table 6. Validation results of the first and the final model

	First model	**Final model (best_5.pt)**
Precision	0.877	0.918
Recall	0.815	0.851
mAP@0.5	0.864	0.887
mAP@0.5:0.95	0.641	0.678
Epochs trained	22	100
Training platform	Google Colab GPU	i9-14900K + RTX A4000
Minutes per epoch	15.4	6.3

The final model beats the first model on every metric (Table 6). Precision rose from 0.877 to 0.918, recall from 0.815 to 0.851, mAP@0.5 from 0.864 to 0.887, and the stricter mAP@0.5:0.95 from 0.641 to 0.678. Two changes explain this: the rebalanced dataset, in which the very large Fire class no longer dominates, and training for the full 100 epochs instead of 22.

Fig. 4 shows that the validation metrics rise quickly during the first 20 epochs and then improve slowly. The validation losses keep decreasing without rising again, so the model does not over-fit. A sharp drop in the training losses after epoch 90 marks the point where mosaic augmentation is switched off and the model sees natural, unmixed images.

One epoch took 6.3 minutes on the workstation, against 15.4 minutes for the first model on Google Colab. The datasets differ, so this is not a pure hardware comparison. It does agree with the team's observation that the two-core Colab CPU could not load and augment images fast enough to keep the GPU busy.

Fig. 5 shows correct detections of fire, smoke, two snakes, a cat and a dog. Smoke and snakes are hard classes because they have no fixed shape and blend into the background; the colour and erasing augmentations help with exactly these cases.

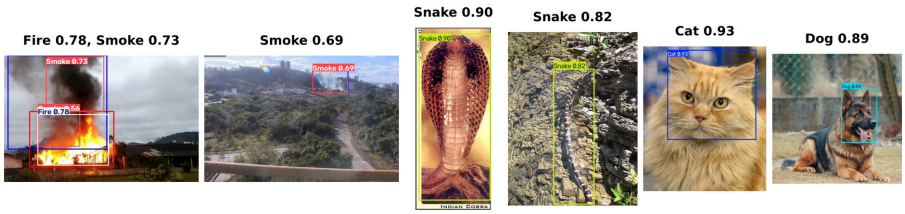


Fig. 5. Detections of the final model at 480×480 on public test images (D-Fire, ImageNet and a sample image). The fire image from the D-Fire example set also contains the dataset's own drawn boxes.

4.2 Edge Deployment

Table 7 gives the frame rates on the Raspberry Pi 5, and Table 8 gives the whole-model latency on the M1 laptop (first model, 640×640 input), which supports the INT8 observation on a second ARM device.

Table 7. Frame rate on the Raspberry Pi 5

Runtime and model	Input	Frame rate
OpenVINO, INT8	—	≈ 0.1 FPS
NCNN, best_5.pt	480×480	≈ 18–20 FPS (display mode)

Table 8. Whole-model latency on the Apple M1 laptop

Engine and device	Precision	ms per frame	FPS
PyTorch, M1 CPU	FP32	103.7	9.6
PyTorch, M1 GPU (Metal)	FP32	13.3	75.1
OpenVINO, M1 CPU	FP32	47.9	20.9
OpenVINO, M1 CPU	FP16	37.8	26.5
OpenVINO, M1 CPU	INT8	54.5	18.3

Why OpenVINO INT8 Was Slow on ARM and NCNN Was Fast. INT8 makes a model faster only when the processor and the runtime both have fast 8-bit integer operations. On the Raspberry Pi 5, OpenVINO computed the quantized layers in floating point. Every layer therefore did its normal work plus the quantize and de-quantize steps, and the system reached only about 0.1 FPS. Our M1 laptop also has an ARM processor and shows the same pattern in Table 8. With OpenVINO, the INT8 model takes 54.5 ms per frame, which is slower than both FP32 (47.9 ms) and FP16 (37.8 ms). FP16 is the fastest CPU option because the ARM vector registers hold twice as many 16-bit values as 32-bit values.

NCNN avoids this problem. Its kernels are written for the ARM NEON vector unit, it uses all cores, and it has no heavy runtime layers. The 480×480 input cuts the work to 56% of the 640×640 work (Eq. (4)), and the smaller number of candidate boxes (4,725 instead of 8,400) shortens post-processing as well. Together these

changes gave about 18–20 FPS, roughly two orders of magnitude faster than the first attempt and fast enough for live monitoring. The runtime and the input size changed at the same time, so these numbers cannot separate the two effects. Sending e-mails from a background thread matters just as much for real-time behaviour, because the frame loop never stops while a message is being sent.

Effect of the Input Resolution on the Edge Device. Reducing the input from 640×640 to 480×480 is a classic trade-off between speed and detail. The work of every convolution grows with the image area, so the network needs only 56% of the operations (Eq. (4)). Layer 1 alone drops from 44.2 to 24.9 million MACs, and the head predicts 4,725 candidate boxes instead of 8,400, which also shortens non-maximum suppression. The price is spatial detail. An object that covers 32×32 pixels at 640×640 covers only 24×24 pixels at 480×480, so very small or distant objects, such as a thin snake or a faint smoke plume far from the camera, become harder to detect. The model was trained and validated at 640×640; its accuracy at 480×480 was not measured separately and is part of future work. For a home camera that watches a room or a doorway at short range most targets are large in the image, which makes the lower resolution an acceptable choice for real-time operation on the Raspberry Pi 5.

CPU Versus GPU on the Whole Model. On the laptop, the M1 GPU runs the whole model in 13.3 ms, 2.8 times faster than the best CPU option (37.8 ms with OpenVINO in FP16) and 7.8 times faster than PyTorch on the CPU. A GPU has many simple arithmetic units, and most YOLO11n layers have enough channels and pixels to keep them busy. The software also matters on the CPU: at the same FP32 precision OpenVINO is 2.2 times faster than PyTorch because it merges layers and uses optimised kernels.

4.3 Layer 1 on the CPU, the GPU and the FPGA

Table 9 and Fig. 6 give the main result of the hardware study. All three produce the 1,638,400 output values of layer 1 for the same image. Four further runs of the CPU measurement gave 5.0–5.3 ms.

Layer 1: Correctness. The simulated circuit produces all 1,638,400 outputs of layer 1, and every one of them is identical to the integer reference model, so the hardware design is correct. The cycle count also matches the design exactly (Eq. (12)). This shows that all 16 MAC units are busy in every clock cycle, with 27 cycles per output position and no stalls. The waveform in Fig. 7 shows this clock by clock, and the feature maps in Fig. 8 show that the outputs are meaningful, with edges and colour regions of the fire image.

The CPU and the GPU both agree with the circuit on 1,637,231 values (99.93%). The other 1,169 values differ by exactly one step, and none differ by more. All three use the same INT8 weights, so the difference comes only from rounding. The circuit uses integers throughout, while the CPU and the GPU compute in floating point and round at the end. A difference of one step out of 256 has no visible effect on the detections.

Table 9. Layer 1 on one 640×640 image: FPGA, CPU and GPU

	FPGA (simulated)	CPU (OpenVINO)	GPU (PyTorch, Metal)
Hardware	16 MACs at 100 MHz	M1, 8 cores	M1, 8 GPU cores
Arithmetic	integer	float32	float32
Time for layer 1	27.6 ms	6.0 ms	5.0 ms (4.9 ms compute only)
Identical to the FPGA output	reference	1,637,231 (99.93%)	1,637,231 (99.93%)
Differ by one step / more	—	1,169 / 0	1,169 / 0

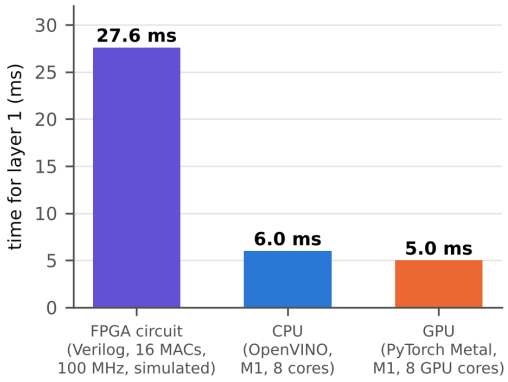


Fig. 6. Time for layer 1 on one 640×640 image on the simulated FPGA circuit, the M1 CPU and the M1 GPU.

Layer 1: Speed. For layer 1 the circuit needs 27.6 ms, the CPU 6.0 ms and the GPU 5.0 ms. That makes the CPU 4.6 times and the GPU 5.5 times faster than the circuit. Eq. (13) explains why. The circuit performs 16 MACs per cycle at 100 MHz, or 1.6 billion MACs per second. Dividing the layer's 44.2 million MACs by the measured times gives effective rates of about 7.4 billion MACs per second on the CPU and 9.0 billion on the GPU, both of which run at gigahertz clock rates with many arithmetic lanes. An FPGA is therefore not automatically faster than a CPU or a GPU. Its speed is the number of parallel MAC units times the clock frequency.

The GPU is only slightly faster than the CPU on this layer, although it is 2.8 times faster on the whole model. Layer 1 has only 3 input channels and the largest feature maps in the network. It reads 1.23 MB and writes 1.64 MB but performs only about 15 MACs per byte moved. A layer like this is limited by memory traffic rather than arithmetic, so the many GPU cores cannot be kept busy. The copies to and from the GPU cost very little here (5.0 ms with copies against 4.9 ms without). The M1 uses one unified memory for the CPU and the GPU, so the image does not have to travel over an external bus as it would to a separate graphics card.

4.4 FPGA Simulation

Table 10 lists the output of the simulation. Fig. 7 shows the waveform of the first output pixel and Fig. 8 the 16 output feature maps of the circuit.

Table 10. Output of the FPGA simulation (Icarus Verilog 13.0)

Quantity	Value
Clock cycles	2,764,827
Output values (320 × 320 × 16)	1,638,400
Multiply-accumulates	44,236,800
Cycles per output pixel	27.000
Time at 100 MHz	27.648 ms
Identical to the integer reference	1,638,400 / 1,638,400 (100%)

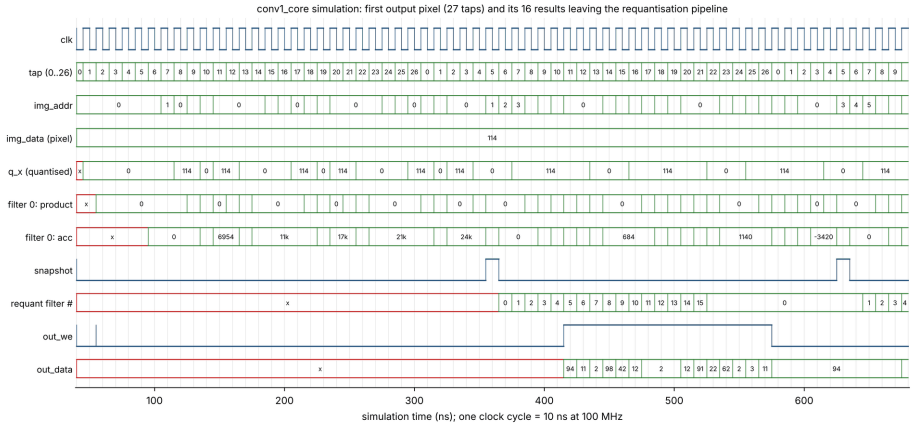


Fig. 7. Simulation waveform of the first output pixel: clock, the 27 taps, the input, one MAC unit and the first outputs leaving the pipeline (one clock cycle = 10 ns at 100 MHz).

Pipelining and Fixed Timing. The circuit reaches exactly 27 cycles per output position because its work overlaps like an assembly line. Each tap passes through address generation, memory read, the input table and weight read, the multiplication and the accumulation, each in its own clock cycle. A new tap therefore enters every cycle while earlier taps are still being processed. When the 27th tap of a position has been accumulated, the 16 sums are copied into a second set of registers (a snapshot), and the MAC units immediately start on the next position. The copied sums then leave one per cycle through the five-stage requantization pipeline and the SiLU table while the MAC units are already busy. Requantization therefore adds no time per pixel, and only the pipeline fill appears as the extra 27 cycles in Eq. (12). The waveform in Fig. 7 shows this: the outputs of the first position appear while the MAC units are already accumulating the second. The schedule is fixed by the hardware, so every frame takes exactly the same number of cycles. The CPU time of the same layer varied between 5.0 and 6.0 ms over five runs.

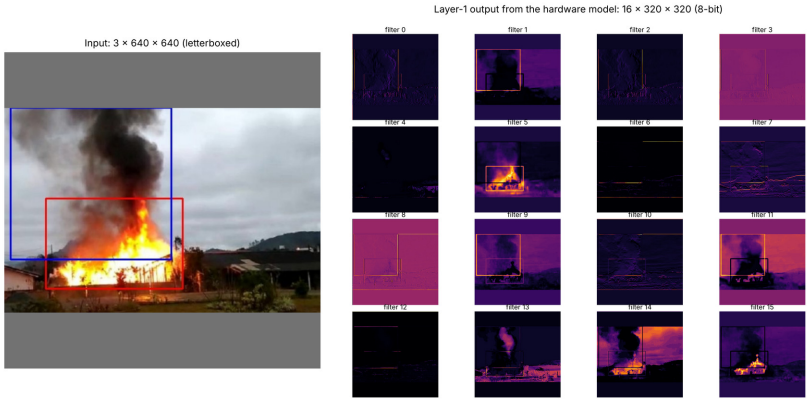


Fig. 8. Input image (letterboxed D-Fire example) and the 16 layer-1 feature maps produced by the circuit.

4.5 Scaling the Circuit (Calculated)

Table 11 applies Eq. (13) to circuits with more MAC units. These values are calculated from the measured cycle count, not simulated.

Table 11. Layer-1 time with more MAC units (calculated, 100 MHz)

MAC units	Cycles per position	640×640 input	480×480 input
16 (our circuit)	27	27.6 ms (simulated)	15.6 ms
48	9	9.2 ms	5.2 ms
144	3	3.1 ms	1.7 ms
432 = 16 × 27	1	1.0 ms	0.6 ms

Making the Circuit Faster. The circuit can be made faster by adding MAC units in the right direction. Adding units across filters does not help, because layer 1 has only 16 filters. Instead, each filter can get 27 MAC units, one for each value of the 3×3×3 window. With 16 × 27 = 432 units, all 27 multiplications of a window happen in the same cycle, and each output position takes one cycle instead of 27. Table 11 shows that layer 1 would then take about 1.0 ms at 640×640, which is faster than the CPU and the GPU, and about 0.6 ms at the 480×480 input used on the Raspberry Pi 5. Such a circuit needs 27 input values per cycle, so it also needs line buffers that keep the last rows of the image on the chip.

We kept 16 MAC units because the design targets a small, low-cost FPGA with only a few dozen hardware multipliers. The 16-unit circuit is proven correct value by value, so the larger version only replicates the same MAC unit. Even at equal speed, an FPGA offers fixed timing (exactly 27 cycles per pixel for every frame), no operating system and potentially much lower power than a GPU [11, 20].

Memory Needed by a Real Circuit. In the simulation, the whole 640×640 image (1.2 MB) is held in a memory model inside the testbench. A low-cost FPGA has far less on-chip memory, so on a real device the pixels would be streamed from the camera or from external memory. Only the three image rows needed by the 3×3 window would be kept on the chip, which is 3 rows × 640 pixels × 3 colours = 5,760 bytes. The 432 weights (432 bytes), the input table (256 bytes) and the SiLU table (64 KB) fit easily. Keeping data on the chip and reusing it is the main design rule of efficient CNN accelerators [11, 20].

System-Level View: Amdahl's Law. Accelerating a single layer has only a limited effect on the whole system. According to the model summary printed on our laptop, the network needs 6.4 GFLOPs per 640×640 frame, or about 3.2 billion MACs, of which layer 1 accounts for 44.2 million (about 1.4%). Measured on its own, layer 1 takes 6.0 ms on the M1 CPU, about 11% of the 54.5 ms for the whole INT8 model. The whole model averages about 59 billion MACs per second, while layer 1 reaches only about 7.4 billion because of its low arithmetic intensity. If a fraction p of the run time is accelerated by a factor k, the overall speed-up is

$$S = \frac{1}{(1 - p) + p/k} \tag{14}$$

With p ≈ 0.11 in Eq. (14), even an infinitely fast layer 1 (k → ∞) would make the whole model only about 1.12 times faster. A useful FPGA accelerator must therefore cover most of the network, and our verified layer-1 circuit is a first step toward that.

4.6 Comparison With Related Work

Compared with the FPGA detectors in Table 2, our circuit is deliberately small. The YOLO design of [23] reaches 1.877 TOPS at 200 MHz on a large Xilinx VC707 board, but only after retraining with binary weights, which cost 2.63 points of mAP. The Angel-Eye flow [21] also quantizes to 8 bits, but it uses its own compiler and a fixed architecture. Our circuit instead takes the INT8 weights exactly as OpenVINO produces them and reproduces the reference result value by value, which makes it a verified starting point for a larger design.

On the CPU side, [15] reported INT8 speed-ups of 1.02–1.56× on an Intel CPU, whereas on our two ARM devices INT8 with OpenVINO was slower than floating point. The benefit of a number format depends on the processor and the runtime, not on the model alone. For edge deployment, [4] ran a pruned fire detector on a Raspberry Pi 4, and [17] accelerated image classification on a Raspberry Pi 4B with an Intel NCS2 stick, which current OpenVINO versions no longer support. Our system runs an eight-class detector at about 18–20 FPS on a Raspberry Pi 5 without an accelerator, using NCNN on the ARM CPU.

4.7 Limitations

The hardware study used the INT8 OpenVINO version of our first model with a 640×640 input. The final model has the same first layer but different weights. Only

layer 1 was implemented as a circuit, and it was tested in simulation, not on an FPGA board. The GPU was used through PyTorch because OpenVINO's GPU plugin supports Intel GPUs only. The Raspberry Pi 5 frame rates are approximate values observed in display mode, and per-class accuracy values are not stored in the checkpoint.

5 Conclusion and Future Scope

We built a smart home hazard and intrusion detection system that runs entirely on a Raspberry Pi 5. A YOLO11n model trained for 100 epochs on a rebalanced dataset reaches a precision of 0.918, a recall of 0.851 and an mAP@0.5 of 0.887. INT8 inference with OpenVINO was not a fast path on ARM processors, with about 0.1 FPS on the Raspberry Pi 5 and slower than FP16 on the M1. NCNN with a 480×480 input gave about 18–20 FPS, and alerts with a snapshot are sent from a background thread.

Our Verilog circuit for the first convolution layer, with 16 parallel MAC units, produced all 1,638,400 outputs correctly in 27 cycles per pixel (27.6 ms at 100 MHz) and agreed with the CPU and the GPU on 99.93% of the values. It was slower than the M1 CPU (6.0 ms) and GPU (5.0 ms) because of its limited parallelism, and a circuit with 432 MAC units would take about 1 ms.

Future work includes the 432-unit version with line buffers, more layers in hardware, running the circuit on an FPGA board, measuring power on all devices, and per-class evaluation on a labelled home-camera test set.

References

1. Elhanashi, A., Essahraui, S., Dini, P., & Saponara, S. (2025). Early fire and smoke detection using deep learning: A comprehensive review of models, datasets, and challenges. *Applied Sciences, 15*(18), Article 10255.
2. Muhammad, K., Ahmad, J., Lv, Z., Bellavista, P., Yang, P., & Baik, S. W. (2019). Efficient deep CNN-based fire detection and localization in video surveillance applications. *IEEE Transactions on Systems, Man, and Cybernetics: Systems, 49*(7), 1419–1434.
3. Casas, E., Ramos, L., Bendek, E., & Rivas-Echeverría, F. (2023). Assessing the effectiveness of YOLO architectures for smoke and wildfire detection. *IEEE Access, 11*, 96554–96583.
4. de Venâncio, P. V. A. B., Lisboa, A. C., & Barbosa, A. V. (2022). An automatic fire detection system based on deep convolutional neural networks for low-power, resource-constrained devices. *Neural Computing and Applications, 34*(18), 15349–15368.
5. Yar, H., Khan, Z. A., Rida, I., Ullah, W., Kim, M. J., & Baik, S. W. (2024). An efficient deep learning architecture for effective fire detection in smart surveillance. *Image and Vision Computing, 145*, Article 104989.
6. Khorramshahi, P., Wu, Z., Wang, T., DeLuccia, L., & Wang, H. (2024). Lightweight delivery detection on doorbell cameras. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* (pp. 6962–6971).

7. Chen, H.-Y., Lin, C.-H., Lai, J.-W., & Chan, Y.-K. (2023). Convolutional neural network-based automated system for dog tracking and emotion recognition in video surveillance. *Applied Sciences, 13*(7), Article 4596.
8. Ghadekar, P., Pradhan, M. R., Swain, D., & Acharya, B. (2024). EmoSecure: Enhancing smart home security with FisherFace emotion recognition and biometric access control. *IEEE Access, 12*, 93133–93144.
9. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779–788).
10. Jocher, G., & Qiu, J. (2024). *Ultralytics YOLO11* (Version 11.0.0) [Computer software]. https://github.com/ultralytics/ultralytics
11. Sze, V., Chen, Y.-H., Yang, T.-J., & Emer, J. S. (2017). Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE, 105*(12), 2295–2329.
12. Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 2704–2713).
13. Liang, T., Glossner, J., Wang, L., Shi, S., & Zhang, X. (2021). Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing, 461*, 370–403.
14. Kozlov, A., Lazarevich, I., Shamporov, V., Lyalyushkin, N., & Gorbachev, Y. (2020). *Neural network compression framework for fast model inference* (arXiv:2002.08679). arXiv. https://doi.org/10.48550/arXiv.2002.08679
15. Jin, Z., & Finkel, H. (2020). Analyzing deep learning model inferences for image classification using OpenVINO. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)* (pp. 908–911).
16. Dagli, R., & Eken, S. (2021). Deploying a smart queuing system on edge with Intel OpenVINO toolkit. *Soft Computing, 25*(15), 10103–10115.
17. Gao, T., & Suto, J. (2025). Acceleration of image classification and object tracking by the Intel Neural Compute Stick 2 with power efficiency evaluation on Raspberry Pi 4B. *Sensors, 25*(6), Article 1794.
18. Raspberry Pi Ltd. (2023). *Raspberry Pi 5* [Product information]. https://pip.raspberrypi.com/categories/892-raspberry-pi-5
19. Tencent. (2017). *ncnn: A high-performance neural network inference framework optimized for the mobile platform* [Computer software]. https://github.com/Tencent/ncnn
20. Chen, Y.-H., Krishna, T., Emer, J. S., & Sze, V. (2017). Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits, 52*(1), 127–138.
21. Guo, K., Sui, L., Qiu, J., Yu, J., Wang, J., Yao, S., Han, S., Wang, Y., & Yang, H. (2018). Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 37*(1), 35–47.
22. Ma, Y., Cao, Y., Vrudhula, S., & Seo, J.-s. (2018). Optimizing the convolution operation to accelerate deep neural networks on FPGA. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 26*(7), 1354–1367.

23. Nguyen, D. T., Nguyen, T. N., Kim, H., & Lee, H.-J. (2019). A high-throughput and power-efficient FPGA implementation of YOLO CNN for object detection. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 27*(8), 1861–1873.
24. Mittal, S. (2020). A survey of FPGA-based accelerators for convolutional neural networks. *Neural Computing and Applications, 32*(4), 1109–1139.
25. Intel Corporation. (2025). *FPGA AI Suite handbook* (Version 2025.3). https://www.intel.com/content/www/us/en/docs/programmable/863373/2025-3/
26. Williams, S. (n.d.). *Icarus Verilog* [Computer software]. https://github.com/steveicarus/iverilog
27. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 248–255).
28. Elfwing, S., Uchibe, E., & Doya, K. (2018). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks, 107*, 3–11.
29. Zheng, Z., Wang, P., Liu, W., Li, J., Ye, R., & Ren, D. (2020). Distance-IoU loss: Faster and better learning for bounding box regression. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 34, No. 7, pp. 12993–13000).
30. Li, X., Wang, W., Wu, L., Chen, S., Hu, X., Li, J., Tang, J., & Yang, J. (2020). Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *Advances in Neural Information Processing Systems* (Vol. 33, pp. 21002–21012).